

Robert Haas

VP, Chief Database Scientist @ EnterpriseDB, PostgreSQL Major Contributor and Committer

Monday, October 14, 2019

Braces Are Too Expensive

PostgreSQL has what's sometimes called a Volcano-style executor, after a system called Volcano, about which Goetz Greafe published several very interesting papers in the early to mid 1990s. PostgreSQL was in its infancy in those days, but many of the concepts in the Volcano papers have made their way into PostgreSQL over the years. It may also be that Volcano took inspiration from PostgreSQL or its predecessors; I'm not entirely sure of the history or who took inspiration from whom. In any case, the Volcano execution model has been thoroughly embedded in PostgreSQL for the entire history of the database system; the first chinks in the armor only started to appear in 2017.

What does it mean to say that PostgreSQL's executor is volcano-style? First of all, it means that the query planner generates a plan that is organized into a sort of tree structure. Second, it means that the system is designed to be extensible, with individual nodes that appear in the plan tree having no datatype-specific knowledge, but rather obtaining their knowledge about specific data types from support functions, which in PostgreSQL are attached to an [operator class](#). Third, it means that the executor functions using a "pull" model. Plan nodes generally support an operation that says "give me the next tuple." The executor repeatedly asks the topmost plan node for the next tuple, and if it's not a leaf node, it asks the nodes beneath it for their next tuples and uses those to compute its own next tuple. Here's a simple example:

```
Nested Loop #1
-> Nested Loop #2
    -> Seq Scan on t1
    -> Index Scan on t2
-> Index Scan on t3
```

In this example, which doesn't exactly match the output that EXPLAIN would produce but is conceptually similar, the topmost plan node is Nested Loop #1. So, to get a tuple to return to the user, the executor will ask Nested Loop #1 for a tuple. Nested Loop #1 will start by asking Nested Loop #2 for a tuple. Nested Loop #2 will ask the Seq Scan on t1 for a tuple. When it gets one, it will return it to Nested Loop #2. Nested Loop #2 will now ask the Index Scan on t2 for a tuple. Assuming it gets one, it will combine the data from the Seq Scan on t1 with the data from the Index Scan on t2 and return the result tuple to Nested Loop #1. Nested Loop #1 will now ask for a tuple from the Index Scan on t3. Assuming it gets one, it will combine the data from that tuple with the one from the tuple produced by Nested Loop #2 to produce a result tuple, which will be sent to the user.

What you may notice here is that the program's control flow has to pass through quite a number of nodes that don't end up doing any real work. When Nested Loop #1 is asked for a tuple, it immediately turns around and passes on a very similar request to Nested Loop #2, which immediately turns around and passes on a very similar request to the Seq Scan on t1. The real work of the Nested Loop happens after those recursive requests complete. Then, the nested loop has to do the work of combining data from its two inputs, and perhaps other work, such as checking join or filter conditions, which I didn't include in this example but which would be present in most real-world examples. It doesn't change the basic picture: all of the interesting work happens after the subordinate nodes have produced tuples.

We could save some effort if we started by calling the Seq Scan on t1, and then the Index Scan on t2, and then, if both returned a tuple, calling the Nested Loop #2 to combine them into an output tuple. Then we'd call the Index Scan on t3 and finally Nested Loop #1. This would involve less transfer of control than the present system.

It's easy to think that perhaps this doesn't matter very much, and in a certain sense that's true. After all, function calls are pretty cheap. If Nested Loop #1 is asked for a tuple, and it turns around and asks Nested Loop #2 for a tuple, the only cost is an extra function call, which is not an enormous number of CPU instructions and generally not the sort of thing that keeps programmers awake at night. Sure, many programmers use inline functions to reduce function-call overhead for things that are simple and highly performance-sensitive, and sometimes that makes a big difference, but often it doesn't. And, in general, almost any complex program is going

About Me

 [Robert Haas](#)

[View my complete profile](#)

Blog Archive

- [2026](#) (6)
- [2025](#) (11)
- [2024](#) (17)
- [2023](#) (7)
- [2022](#) (5)
- [2021](#) (3)
- [2020](#) (5)
- ▼ [2019](#) (7)
 - ▼ [October](#) (1)
 - [Braces Are Too Expensive](#)
 - [September](#) (1)
 - [June](#) (1)
 - [February](#) (1)
 - [January](#) (3)
- [2018](#) (5)
- [2017](#) (6)
- [2016](#) (6)
- [2015](#) (4)
- [2014](#) (11)
- [2013](#) (5)
- [2012](#) (14)
- [2011](#) (41)
- [2010](#) (46)

to be constantly calling from and returning from functions, and in most cases the overhead from doing so is only a small percentage of the program's runtime.

However, while the cost is low, it's not zero. Calling a function often means saving CPU registers that the function might clobber to the stack, and restoring them when the function returns. There can be other kinds of overhead as well, such as the need to reload from memory data that the function call might have overwritten (but in reality probably didn't). If the function does a lot of work, these kinds of overhead aren't material, but if it does very little, it can actually be significant. You can see this overhead - at least to some degree - by using a tool that can do instruction-level profiling, such as `perf`. `perf annotate` will attribute the cost of each instruction to some line of code, and that sometimes makes the function call overhead visible. You'll sometimes see, for example, that the opening curly brace of some function consumes a noticeable chunk of CPU time. You might wonder how that's possible, since that curly brace seems not to be executable code. In fact, `perf` (and probably other tools) attribute to that line of code, or sometimes to the first line of code inside the function, the overhead of setting up that function's stack frame - or in other words, a portion of the function call overhead.

In 2017, Andres Freund, a longtime PostgreSQL hacker who is now a colleague of mine at EnterpriseDB, tackled a problem closely related to this one. As it turns out, not only are PostgreSQL plans represented as a tree of executable nodes, but so are PostgreSQL expressions. For example, if you try to select all rows from a table where `foo = bar + baz`, you actually get a tree with `=` at the root, `foo` on one side, and a subtree with `+` on the other side. That subtree, in turn, has `bar` on one side and `baz` on the other. Up until that work, execution recursed down that tree of expression nodes in a manner very similar to what it still does today with plan nodes. Andres's work changed it around so that it transformed the expression tree into a series of steps that are executed one after another, like a sort of dynamically constructed program.

How much did it help? See https://www.pgcon.org/2017/schedule/attachments/462_jit-pgcon-2017-05-25.pdf#page=7. What you'll find there is that it made at least a small difference for practically all of the TPC-H queries, and for some queries, like TPC-H Q1, it made a really large difference. This surprised me quite a bit at a time, because the actual lines of executable code being run for any given query really didn't change that much as a result of that work. What did change is the number of times we had to enter and exit functions. In other words, a surprising amount of the cost was hidden in a part of the code that I would never have thought to optimize: the braces! (I apologize to anyone who was hoping this post was going to be about orthodontic expenses.)

Andres is now thinking about optimizing plan trees using a similar technique. I don't think we'll see similar speed-ups, because a plan operator like `Seq Scan` or `Nested Loop` is considerably more expensive than an expression operator like `+` or `=`, so the function call overhead is less significant as a percentage of the total runtime. Still, there will probably be some performance benefit -- and perhaps other benefits as well, but I'll leave those for a separate blog post, as this one grows long.

Posted by [Robert Haas](#) at 2:39 PM

Labels: [postgresql](#)

2 comments:



Colin 't Hart October 17, 2019 7:52 AM

How does this relate to the JIT compilation work? I remember that when reading the presentation on it the author talked about converting Postgres to a push-based execution model. But I didn't understand all the nuances back then. After reading this I should go and read that presentation again.

[Reply](#)

▼ Replies



Robert Haas October 20, 2019 8:35 PM

As I understand it, this kind of work is necessary for JIT to produce good results. You need to have a substantial chunk of code to compile, and if you don't first transform the recursive tree traversal into an iterative program, you just have one line of code: the first recursive call to the top-level node.

[Reply](#)



Enter Comment



finanzkompass-tips

Kinder erben

Schulden – wenn

Eltern vergessen

vor dem

